Chapter 09

Automation & Scripting

Episode 9.01

Episode title: **Infrastructure as Code**

Objective: 4.1

Infrastructrure as Code Overview

Infrastructure as Code (IaC) concepts

Orchestration

Configuration management

Unattended Deployment

Kickstart automates OS installation

`cloud-init` configures systems at first boot

Used for consistent provisioning

Configuration Management vs Orchestration

Configuration management concepts

Orchestration concepts

Idempotency and desired state

Common IaC Tools

Ansible and Puppet roles

Terraform and infrastructure provisioning

Kubernetes and container orchestration

Ansible Components (Exam Focus)

Playbooks define automation workflows

Inventory lists managed hosts

Modules perform tasks

Ad hoc commands run one-time actions

Facts collect system information

Collections bundle content

Ansible is agentless

Puppet Architecture & Components

Classes define desired configuration

Modules provide reusable logic

Facts describe system state

Certificates secure communication

Puppet commonly uses agents

OpenTofu Infrastructure & Orchestration

Terraform-compatible IaC tool

Providers connect to APIs

Resources define infrastructure

State tracks deployed objects

Kubernetes Objects

Pods run containers

Deployments manage desired state

Services provide networking

ConfigMaps store configuration

Secrets store sensitive data

Volumes persist data

Docker Swarm Concepts

Services define workloads

Nodes run containers

Tasks are running containers

Networks connect services

Swarm supports scaling

Docker and Podman Compose

Compose files define applications

Start and stop environments

View logs for services

Used for local orchestration

Benefits, Risk, and Best Practices of IaC

Benefits of IaC

- Consistency
- Repeatability
- Scalability
- Improved security posture

Benefits, Risk, and Best Practices of IaC

Benefits of IaC

Risks and misconfigurations

- Errors in code

Best practices

Benefits, Risk, and Best Practices of IaC

Benefits of IaC

Risks and misconfigurations

Best practices

- Store IaC in secure version control systems
- Separate secrets from code
- Use testing and validation tools
- Apply least privilege to credentials

CI/CD, GitOps, and DevSecOps

CI/CD automates testing and deployment

Shift-left testing finds issues earlier

GitOps uses Git as source of truth

DevSecOps integrates security

Infrastructure as Code Concepts (DEMO)

Demonstrate Ansible playbook structure

Show Terraform plan vs. apply

Kubernetes manifest overview

Episode 9.02

Episode title: **Bash scripting**

Objective: 4.2

Bash Scripting Overview

Why Bash scripting matters
Automation and repeatability

Script Structure, Variables, & Exit Codes

Shebang and execution

Variables and positional parameters

Exit codes

Shell Expansion & Substitution

Parameter expansion uses `${var}`

Command substitution captures output

Subshells run isolated commands

Conditionals & Logical Testing

If/elif/else

File, string, numeric tests

Logical operators

Tests, Comparisons, and Regex

Numeric comparisons

-eq -gt -lt

String comparisons

== and !=

File tests check files and directories

Regex matching uses double brackets

Functions, Input, & Output Handling

for/while loops

Functions

Redirection and pipes

Bash Scripting Basics (DEMO)

Create and run scripts

Use variables, conditionals, and loops

Debug and test scripts

Episode 9.03

Episode title: **Bash Scripting Demo**

Objective: 4.2

Episode 9.04

Episode title: **Python for Admins**

Objective: 4.3

Python for Admins Overview

Why Python matters for Linux administrators

Python vs. Bash decision-making

Python Virtual Environments

Isolate Python dependencies

Prevent system conflicts

Commonly created with `venv`

Python Script Structure, Execution, & Variables

Python interpreter and shebang

Variables and data types

Input and output

Python Data Types

Boolean values

Integers and floats

Strings

Lists and dictionaries

Control Structures, Functions, & Error Handling

If/elif/else statements

for/while loops

Functions and exceptions

Files, System Interaction, & Admin Use Cases

Reading and writing files

Running system commands

Real-world admin scenarios

Python for Admins (DEMO)

Build a complete admin script

Use variables, functions, loops, and errors

Execute system commands safely

Episode 9.05

Episode title: **Python for Admins Demo**

Objective: 4.3

Episode 9.06

Episode title: **Git & Version Control**

Objective: 4.4

Git & Version Control Overview

Why version control matters for Linux administrators

Git fundamentals

Safe workflows

Git Commands Exam Coverage

Initialize and clone repositories

Add and commit changes

Branch and merge code

Pull and push updates

Reset and stash changes

Use `.gitignore`

Repositories, Commits, & History

Git repositories

Commits as snapshots

Viewing and understanding history

Branching, Merging, & Change Isoloation

Purpose of branches

Merging changes safely

Avoiding configuration breakage

Remotes, Push/Pull, & Team Workflows

Remote repositories

`push | pull` operations

Collaboration patterns

Git for Linux Administrators (DEMO)

Initialize and manage a repository

Track, commit, branch, and merge changes

Review history and recover safely

Episode 9.07

Episode title: **AI & Best Practices**

Objective: 4.5

AI & Best Practices Overview

AI and automation in Linux administration

Responsible usage and guardrails

AI-Assisted Automation: Use Cases & Boundaries

Common AI use cases for administrators

Where AI adds value

Clear operational boundaries

AI Use Cases for Linux Admins

Generate scripts and code

Create regular expressions

Generate infrastructure as code

Review security and compliance

Improve documentation

Best Practices for AI Usage

Validation and testing

Version control and change management

Least privilege and separation of environments

Security, Privacy, & Ethical Considerations

Data sensitivity and confidentiality

Security posture and controls

Compliance and policy alignment

AI in Operational Decision-Making

Supporting analysis, not execution

Incident response
and troubleshooting

Continuous improvement

Exam Tips & Summary

Safety over speed

Human oversight is mandatory